

Calcul Parallèle

TP : Partie 2

Thomas STAVIS
(thomas.stavis@ens-lyon.fr)

Jonathan Rouzaud-Cornabas
(jonathan.rouzaud-cornabas@inria.fr)

Submitting Same rule as TP part 1.

Notation Part 2 of this TP is about 28 points. The score for this part will be *minimum*(20, your score at the part)/20. The final grade of TP will be an average of the grades of the 2 parts.

2 Matrix multiplication (/28)

In this section we will use parallelism to increase the execution speed, and thus the speed-up, of matrix multiplication. Particular importance will be given to reasoning (calculation errors, as long as the reasoning is correct and the result non-aberrant, are not penalized).

2.1 Multi-parallelism (/14)

In part 1 of this TP, we discussed multi-threading, but this technique is limited (there are not an infinite number of threads in a computer). However, there are other types of parallelism we can mix to achieve better speed-up (we don't have the time here, but I recommend you to search by yourself what are these tools) :

- AVX instructions (CPU)
- pipelining
- multi-cores (physical)
- multi-process
- multi/hyper-threading
- GPU
- multi-CPU/GPU
- multi-computers

Important : For the purposes of the TP, all matrices will be square and of size $2m$ with $m \in \mathbb{N}$ (thus contain $(2^m)^2$ elements), and contain integers less than 2^{16} coded on 64 bits. Here is the "classic" algorithm of matrix multiplications, its complexity is about $7n^3 + 7n^2$, so in $\mathcal{O}(n^3)$.

Algorithm 1: Multiplication matrices séquentiel

Data: A, B matrices, n taille des matrices

```
1  $M \leftarrow Mat[n][n]$ 
  for  $i = 0$  to  $n - 1$  do
    for  $j = 0$  to  $n - 1$  do
       $sum \leftarrow 0$ 
5      for  $k = 0$  to  $n - 1$  do
         $sum \leftarrow sum + A[i][k] \times B[k][j]$ 
       $M[i][j] \leftarrow sum$ 
8 return  $M$ 
```

Question 1 (/1)

Suppose that we have access to a single-core CPU with a frequency of 1 GHz (1 billion operations are performed per second), for 2 matrices of size $n = 1\,024$, what would be the execution time, at the nearest second, of their multiplication? Same question for a matrix of size $n = 8\,192$?

Question 2 (/3)

Using only multi-threading, explain how the Algorithm 1 can be parallelized. What would be the speed-up if you used 8 threads for a matrix size of $n = 8\,192$? Is there an ideal number of threads to use for this execution?

Question 3 (/10)

You have at your disposal :

- several computers, whose motherboards have 1, 2 or 4 CPU sockets, and 1 or 2 GPU sockets
- several CPUs, including 1, 2, 4, 8 or 32 cores at 3 GHz on 64 bits processors, with 2 threads by core
- several GPUs, comprising 1024, 2048, 4096 or 8192 cores single-thread 1 GHz on 64 bits
- the following algorithms :
 - multiplication of sequential matrices of complexity $7n^{\log_2(7)} - 6n^2$
 - multiplication of matrices by block (with algorithms for division into 4 sub-matrices and merging 4 sub-matrices of complexity $7n \times \log_2(n)^2$ (Well, it resembles sorting fusion...))
 - compression without integer size (64 bits to 32 or 16 bits)

With what you have available, give a method using the different ways to parallelize matrix multiplication to decrease execution time. What time would it take for 2 matrices of size $n = 8\,192$? Determine the speed-up compared to the Algorithm 1. (There is not a single good answer, it's your reasoning that counts)

2.2 Space constraints (/7)**Question 1 (/1)**

How many bytes (1 byte = 8 bits) of storage are needed to store a matrix of size n ?

Question 2 (/1)

If all matrices were loaded in RAM, how many giga bytes of RAM would be needed to execute the Algorithm 1, for matrices of size $n = 8\,192$?

Question 3 (/5)

It is actually not necessary to load the entire matrices into RAM. We can, for example, load only the line i of the matrix A , the column j of the matrix B , and only one element of M . Following this reasoning and using the method you gave at the end of Section 2.1, calculate the minimum RAM needed for each computer, for a multiplication of matrices of size $n = 8\,192$.

2.3 Communication constraints (/7)

Question 1 (/1)

Suppose we use a data bus with a throughput of 240 MB/s (with a latency of 0 ms) between RAM and persistent storage. What is the time needed to load the A and B matrices of size $n = 8\,192$ in RAM? Same question for loading and unloading a row and a column in RAM.

Question 2 (/1)

Which method from Question 1 gives a smaller total loading time? Knowing that RAM loading can be done in parallel with the execution of instructions, which method is the most efficient?

Question 3 (/1)

To transfer data between 2 computers, we have a connection of 1 GB/s, with a latency of 200 ms. Calculate the time it takes to transmit a matrix of size $n = 2\,048$ from one computer to another.

Question 4 (/4)

Suppose that :

- each computer can use in parallel 4 inter-computer connections with a rate of 1 Gb/s and latency 100 ms
- the CPU-RAM buses have a throughput of 240 Mb/s with a latency of 0 ms
- the GPU-RAM buses have a throughput of 2 Gb/s with 0 ms of latency of 0 ms
- each computer has 2 TB of RAM (same for the GPUs)

For the multiplication of 2 matrices of size $n = 8\,192$, with your method defined at the end of Section 2.1, give approximately the execution time of these 2 matrices taking into account the communications. Calculate the speed-up again.